Release Document

Status: Completed

Whenever developers push code changes or manually select a service for deployment, the CI/CD pipeline performs upstream quality gates, builds Docker images, runs image security scans, publishes approved images to DockerHub, and then proceeds with deployment operations.

This release document is written in an internal operational format and is intended to be executed step-by-step.

SECTION 1: Infrastructure Setup (Kubernetes Cluster + NFS + HAProxy + SonarQube)

This section defines the complete baseline infrastructure required to run the E-Commerce platform on Kubernetes, including cluster provisioning, shared storage, load balancing, and the upstream code-quality server used by CI quality gates.

1.1 Architecture Overview

```
┌──────────────────────────────────────────┐
│ GITHUB ACTIONS (CI/CD)                   │
│                                          │
│ ┌──────────┐  ┌──────────┐  ┌──────────┐ │
│ │ Build    │→ │ SonarQube │→ │ Docker   │ │
│ │ & Test   │  │ Scan     │  │ Build    │ │
│ └──────────┘  └──────────┘  └──────────┘ │
└──────────────────────────────────────────┘
                                │
                                │
                                │ kubectl apply
                                ▼
┌──────────────────────────────────────────────────────────────────┐
│ KUBERNETES CLUSTER                                                 │
│                                                                    │
│ ┌──────────────────────────────────────────┐                      │
│ │ MASTER NODE (t3.medium)                  │                      │
│ │                                          │                      │
│ │ • kube-apiserver   • kube-scheduler      │                      │
│ │ • kube-controller  • etcd                │                      │
│ │ • CoreDNS          • Calico CNI          │                      │
│ └──────────────────────────────────────────┘                      │
│                                                                    │
│ ┌──────────▼──────┐                  ┌──────────▼──────┐          │
│ │ WORKER-1        │                  │ WORKER-2        │          │
│ │ (t3.medium)     │                  │ (t3.medium)     │          │
│ │                 │                  │                 │          │
│ │ • kubelet       │                  │ • kubelet       │          │
│ │ • kube-proxy    │                  │ • kube-proxy    │          │
│ │ • containerd    │                  │ • containerd    │          │
│ │                 │                  │                 │          │
│ │ Pods:           │                  │ Pods:           │          │
│ │ - frontend      │                  │ - frontend      │          │
│ │ - backend-api   │                  │ - backend-api   │          │
│ └─────────────────┘                  └─────────────────┘          │
└──────────────────────────────────────────────────────────────────┘
            │                                  │
            │ NodePort:30080/30081             │ NFS Mount
            ▼                                  ▼
┌────────────────────────┐           ┌────────────────────────┐
│ NFS SERVER + HAProxy   │           │ NFS Storage            │
│ (t3.medium)            │           │                        │
│                        │           │ /srv/nfs/k8s-pvs/      │
```

```
│ HAProxy:                  │          │ ├── products/            │
│ Port 80  → Workers        │          │ ├── orders/              │
│ Port 443 → Workers        │          │ ├── users/               │
│ Port 8404→ Stats          │          │ ├── logs/                │
│                           │          │ └── shared/              │
│ NFS: Port 2049            │          │                         │
└───────────────────────────┘          └─────────────────────────┘
            │
            │
            ▼
        ┌──────────┐
        │ USERS    │
        │(Internet) │
        └──────────┘


┌────────────────────────┐
│ SONARQUBE SERVER       │
│ (t3.medium)            │
│                        │
│ Port: 9000             │
│ H2 Embedded Database    │
│ Code Quality Analysis   │
└────────────────────────┘
```

1.2 Infrastructure Inventory

#	Server	Role	Instance Type	Private IP	OS	Key Ports
---:	---	---	---	---	---	---
1	master-node	K8s Control Plane	t3.medium	172.31.1.10	Ubuntu 22.04 LTS	6443, 2379-2380, 10250-10252
2	worker-node-1	K8s Worker	t3.medium	172.31.1.11	Ubuntu 22.04 LTS	10250, 30000-32767
3	worker-node-2	K8s Worker	t3.medium	172.31.1.12	Ubuntu 22.04 LTS	10250, 30000-32767
4	nfs-haproxy	NFS + Load Balancer	t3.medium	172.31.1.13	Ubuntu 22.04 LTS	2049, 80, 443, 8404
5	sonarqube	Code Quality	t3.medium	172.31.1.14	Ubuntu 22.04 LTS	9000

Note: Replace all example IPs with your actual AWS EC2 private IPs throughout this document.

1.3 Pre-requisites & AWS Security Groups

Configure the following Security Groups in AWS Console before starting:

SG: sg-k8s-master

Inbound Rules:

- TCP `6443` → `172.31.0.0/16` (K8s API Server)
- TCP `2379-2380` → `172.31.0.0/16` (etcd)
- TCP `10250-10252` → `172.31.0.0/16` (Kubelet/Controller)
- TCP `22` → Your IP (SSH)

SG: sg-k8s-worker

Inbound Rules:

- TCP `10250` → `172.31.0.0/16` (Kubelet)
- TCP `30000-32767` → `172.31.0.0/16` (NodePort Services)
- TCP `22` → Your IP (SSH)

SG: sg-nfs-haproxy

Inbound Rules:

- TCP `2049` → `172.31.0.0/16` (NFS)
- TCP `111` → `172.31.0.0/16` (RPC portmapper)
- TCP `80` → `0.0.0.0/0` (HTTP via HAProxy)
- TCP `443` → `0.0.0.0/0` (HTTPS via HAProxy)
- TCP `8404` → Your IP (HAProxy Stats)
- TCP `22` → Your IP (SSH)

SG: sg-sonarqube

Inbound Rules:

- TCP `9000` → `0.0.0.0/0` (SonarQube UI)
- TCP `22` → Your IP (SSH)

1.4 Phase 1: Kubernetes Cluster Setup

1.4.1 Setup Order

- Step 1 → All 3 Nodes (Common Setup)
- Step 2 → Master Node Only (`kubeadm init`)
- Step 3 → Worker Nodes (`kubeadm join`)

1.4.2 Common Setup — Run on ALL 3 Nodes (Master + Worker1 + Worker2)

SSH into each node and run the following commands one by one.

Step 1: Update System Packages

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Disable Swap

Kubernetes requires swap to be disabled. If swap is enabled, kubelet will not start.

```
# Disable swap immediately (for current session)
sudo swapoff -a

# Disable swap permanently (survives reboot)
# This comments out the swap line in /etc/fstab
```

```
sudo sed -i '/ swap / s/^/#/' /etc/fstab

# Verify swap is completely off
free -h
```

Expected Output:

```
              total        used        free
Swap:            0B          0B          0B
```

Step 3: Load Required Kernel Modules

These modules are required for Kubernetes networking to work properly.

```
# Create the modules config file
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

# Load the modules immediately (without reboot)
sudo modprobe overlay
sudo modprobe br_netfilter

# Verify modules are loaded
lsmod | grep br_netfilter
lsmod | grep overlay
```

Expected Output:

```
br_netfilter 28672 0
bridge 307200 1 br_netfilter
overlay 151552 0
```

Step 4: Configure Kernel Network Parameters

These settings enable iptables to see bridged traffic and enable IP forwarding.

```
# Write the sysctl configuration
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

# Apply settings immediately without reboot
sudo sysctl --system

# Verify the settings are applied
```

```
sysctl net.bridge.bridge-nf-call-iptables
sysctl net.ipv4.ip_forward
```

Expected Output:

```
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
```

Step 5: Install Containerd (Container Runtime)

Kubernetes requires a container runtime. We use `containerd`, which is the industry standard.

```
# Install required packages
sudo apt install -y ca-certificates curl gnupg lsb-release apt-transport-https

# Create the directory for GPG keys
sudo install -m 0755 -d /etc/apt/keyrings

# Add Docker's official GPG key (containerd is distributed via Docker repo)
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository to apt sources
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Update and install containerd
sudo apt update
sudo apt install -y containerd.io

# Verify containerd installation
containerd --version
```

Step 6: Configure Containerd

This is critical. Without `SystemdCgroup` enabled, pods may crash or behave unpredictably.

```
# Generate the default configuration file
sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee /etc/containerd/config.toml > /dev/null

# Enable SystemdCgroup - REQUIRED for Kubernetes
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.
toml
```

```
# Verify the change was applied correctly
grep -i "SystemdCgroup" /etc/containerd/config.toml

# Restart containerd to apply new configuration
sudo systemctl restart containerd
sudo systemctl enable containerd

# Verify containerd is running
sudo systemctl status containerd
```

Expected Output:

```
SystemdCgroup = true
```

Step 7: Install Kubernetes Components

We install:

- `kubeadm` — bootstraps the cluster
- `kubelet` — runs on every node, manages pods
- `kubectl` — CLI tool to interact with cluster

```
# Add Kubernetes official GPG key
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

# Add Kubernetes repository
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

# Update package list
sudo apt update

# Install kubelet, kubeadm, kubectl
sudo apt install -y kubelet kubeadm kubectl

# Pin the versions to prevent accidental upgrades (stability requirement)
sudo apt-mark hold kubelet kubeadm kubectl

# Verify all three are installed
kubeadm version
kubectl version --client
kubelet --version
```

Step 8: Install NFS Client

Required on all worker nodes so they can mount NFS volumes from the NFS server.

```
sudo apt install -y nfs-common
```

Step 9: Set Hostname and Update /etc/hosts

On Master Node only:

```
sudo hostnamectl set-hostname master-node
```

On Worker Node 1 only:

```
sudo hostnamectl set-hostname worker-node-1
```

On Worker Node 2 only:

```
sudo hostnamectl set-hostname worker-node-2
```

On ALL 3 nodes — add entries to `/etc/hosts`:

```
sudo bash -c 'cat >> /etc/hosts << EOF
# Kubernetes Cluster Nodes
172.31.1.10 master-node
172.31.1.11 worker-node-1
172.31.1.12 worker-node-2
172.31.1.13 nfs-server
EOF'
cat /etc/hosts
```

1.4.3 Master Node Setup — Run ONLY on Master Node

SSH into the Master Node:

```
ssh -i "your-key.pem" ubuntu@<master-node-public-ip>
```

Step 1: Initialize the Kubernetes Cluster

```
sudo kubeadm init \
  --pod-network-cidr=10.244.0.0/16 \
  --apiserver-advertise-address=172.31.1.10 \
  --kubernetes-version=v1.29.0 \
  --control-plane-endpoint=172.31.1.10:6443
```

What each flag means:

Flag	Purpose
`--pod-network-cidr`	IP range for pod network (must match Calico config)

Flag	Purpose
`--apiserver-advertise-address`	Master node's private IP
`--kubernetes-version`	Pin to a specific version
`--control-plane-endpoint`	Endpoint for the API server

Important: Copy and save the `kubeadm join` command from the output. You will need it for the worker nodes.

Step 2: Configure kubectl Access

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

kubectl get nodes
```

Expected Output (before CNI):

```
NAME         STATUS     ROLES           AGE   VERSION
master-node  NotReady   control-plane   1m    v1.29.0
```

Step 3: Install Calico CNI (Pod Network Plugin)

Without a CNI plugin, pods cannot communicate with each other.

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/mani
fests/calico.yaml

kubectl get pods -n kube-system -w
```

Wait until pods show `Running`, then verify nodes:

```
kubectl get nodes
```

Expected Output:

```
NAME         STATUS   ROLES           AGE   VERSION
master-node  Ready    control-plane   5m    v1.29.0
```

Step 4: Generate Join Token (In Case You Need It Later)

Tokens expire in 24 hours. If the join command expires, generate a new one:

```
kubeadm token create --print-join-command
```

1.4.4 Worker Nodes Setup — Run on Worker-1 and Worker-2

SSH into each worker node:

```
# Terminal 1 - Worker Node 1
ssh -i "your-key.pem" ubuntu@<worker-1-public-ip>

# Terminal 2 - Worker Node 2
ssh -i "your-key.pem" ubuntu@<worker-2-public-ip>
```

Step 1: Join the Cluster

Use the join command from `kubeadm init` output:

```
sudo kubeadm join 172.31.1.10:6443 --token abcdef.1234567890abcdef \
  --discovery-token-ca-cert-hash sha256:abc123def456...
```

1.4.5 Verify Cluster — Run on Master Node

```
kubectl get nodes -o wide
```

Expected Output:

```
NAME           STATUS   ROLES           AGE   VERSION   INTERNAL-IP   OS-IMAGE
master-node    Ready    control-plane   10m   v1.29.0   172.31.1.10    Ubuntu 22.04
worker-node-1  Ready    <none>          3m    v1.29.0   172.31.1.11    Ubuntu 22.04
worker-node-2  Ready    <none>          2m    v1.29.0   172.31.1.12    Ubuntu 22.04
```

Label worker nodes for identification:

```
kubectl label node worker-node-1 node-role.kubernetes.io/worker=worker
kubectl label node worker-node-2 node-role.kubernetes.io/worker=worker
kubectl get nodes
```

Verify system pods:

```
kubectl get pods -n kube-system
kubectl cluster-info
```

1.5 Phase 2: NFS Server + HAProxy Setup

Both NFS and HAProxy are installed on the same EC2 instance (`t3.medium`).

1.5.1 SSH into NFS + HAProxy Server

```
ssh -i "your-key.pem" ubuntu@<nfs-haproxy-public-ip>
```

1.5.2 Part A: NFS Server Setup

Step 1: Update System

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install NFS Server Package

```
sudo apt install -y nfs-kernel-server nfs-common
sudo systemctl status nfs-kernel-server
```

Step 3: Create NFS Export Directories

We create separate directories for each e-commerce workload category:

```
# Create base directory
sudo mkdir -p /srv/nfs/k8s-pvs

# Create service-specific directories
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-products     # Product images & catalog dat
a
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-orders       # Order records
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-users        # User uploads & avatars
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-logs         # Application logs
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-db-backup    # Database backups
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-shared       # Shared assets & config

# Set permissions
sudo chown -R nobody:nogroup /srv/nfs/k8s-pvs
sudo chmod -R 777 /srv/nfs/k8s-pvs

# Verify directory structure
ls -la /srv/nfs/k8s-pvs/
```

Expected Output (structure):

- `ecommerce-db-backup`
- `ecommerce-logs`
- `ecommerce-orders`
- `ecommerce-products`
- `ecommerce-shared`
- `ecommerce-users`

Step 4: Configure NFS Exports

```
sudo nano /etc/exports
```

Add the following configuration:

```
# /etc/exports - NFS Export Configuration
# Format: directory client(options)
# Replace 172.31.0.0/16 with your actual VPC CIDR
/srv/nfs/k8s-pvs/ecommerce-products    172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-orders      172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-users       172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-logs        172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-db-backup   172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-shared      172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
```

NFS Export Options Explained:

Option	Description
`rw`	Allow read and write operations
`sync`	Write data to disk before replying (data safe)
`no_subtree_check`	Disables subtree checking, improves reliability
`no_root_squash`	Allows root on clients to write as root on server

Step 5: Apply NFS Export Configuration

```
sudo exportfs -arv
```

Expected Output (example):

- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-products`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-orders`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-users`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-logs`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-db-backup`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-shared`

Step 6: Start and Enable NFS Server

```
sudo systemctl enable nfs-kernel-server
sudo systemctl restart nfs-kernel-server
sudo systemctl status nfs-kernel-server
```

Step 7: Verify NFS Server

```
sudo exportfs -v
sudo ss -tlnp | grep 2049
showmount -e localhost
```

Expected Output of `showmount -e localhost`:

- `/srv/nfs/k8s-pvs/ecommerce-shared 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-db-backup 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-logs 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-users 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-orders 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-products 172.31.0.0/16`

Step 8: Test NFS from Kubernetes Worker Nodes

On each worker node:

```
# Check what NFS server is exporting
showmount -e 172.31.1.13

# Test mounting an NFS share
sudo mount -t nfs 172.31.1.13:/srv/nfs/k8s-pvs/ecommerce-products /mnt
ls /mnt
sudo umount /mnt
```

If the mount succeeds, NFS is working correctly.

1.5.3 Part B: HAProxy Setup

HAProxy acts as a load balancer, distributing incoming user traffic across both Kubernetes worker nodes.

Step 1: Install HAProxy

```
sudo apt install -y haproxy
haproxy -v
```

Step 2: Backup Default Configuration

```
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
```

Step 3: Configure HAProxy

```
sudo nano /etc/haproxy/haproxy.cfg
```

Replace the entire contents with:

```
#---------------------------------------------------------------------
# Global Settings
#---------------------------------------------------------------------
global
  log /dev/log local0
  log /dev/log local1 notice
  chroot /var/lib/haproxy
  stats socket /run/haproxy/admin.sock mode 660 level admin
  stats timeout 30s
  user haproxy
  group haproxy
  daemon

  # SSL configuration
  ca-base /etc/ssl/certs
  crt-base /etc/ssl/private
  ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256
  ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

#---------------------------------------------------------------------
# Default Settings
#---------------------------------------------------------------------
defaults
  log global
  mode http
  option httplog
  option dontlognull
  option forwardfor
  option http-server-close
  timeout connect 5000ms
  timeout client 50000ms
  timeout server 50000ms
  errorfile 400 /etc/haproxy/errors/400.http
  errorfile 403 /etc/haproxy/errors/403.http
  errorfile 408 /etc/haproxy/errors/408.http
  errorfile 500 /etc/haproxy/errors/500.http
  errorfile 502 /etc/haproxy/errors/502.http
  errorfile 503 /etc/haproxy/errors/503.http
  errorfile 504 /etc/haproxy/errors/504.http

#---------------------------------------------------------------------
# HAProxy Statistics Dashboard
# Access: http://<haproxy-ip>:8404/stats
#---------------------------------------------------------------------
frontend stats
  bind *:8404
  stats enable
  stats uri /stats
  stats refresh 10s
  stats auth admin:StrongPassword@123
  stats admin if LOCALHOST

#---------------------------------------------------------------------
# E-Commerce HTTP Frontend (Port 80)
#---------------------------------------------------------------------
frontend ecommerce_http_frontend
  bind *:80
  mode http

  # Security headers
  http-response set-header X-Frame-Options SAMEORIGIN
  http-response set-header X-Content-Type-Options nosniff
  http-response set-header X-XSS-Protection "1; mode=block"

  # Path-based routing using ACLs
  acl is_api path_beg /api
  acl is_admin path_beg /admin

  # Route API traffic to API backend
```

```
  use_backend ecommerce_api_backend if is_api

  # Route all other traffic to frontend
  default_backend ecommerce_frontend_backend

#---------------------------------------------------------------------
# Frontend Backend Pool
# Worker nodes expose frontend on NodePort 30080
#---------------------------------------------------------------------
backend ecommerce_frontend_backend
  mode http
  balance roundrobin
  option httpchk GET /health
  http-check expect status 200

  # Sticky sessions to keep shopping cart consistent
  cookie SERVERID insert indirect nocache

  server worker-1 172.31.1.11:30080 check cookie w1 inter 5s fall 3 rise 2
  server worker-2 172.31.1.12:30080 check cookie w2 inter 5s fall 3 rise 2

#---------------------------------------------------------------------
# API Backend Pool
# Worker nodes expose API on NodePort 30081
#---------------------------------------------------------------------
backend ecommerce_api_backend
  mode http
  balance roundrobin
  option httpchk GET /api/health
  http-check expect status 200

  server worker-1 172.31.1.11:30081 check inter 5s fall 3 rise 2
  server worker-2 172.31.1.12:30081 check inter 5s fall 3 rise 2
```

Step 4: Validate HAProxy Configuration

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

Expected Output:

```
Configuration file is valid
```

Step 5: Start and Enable HAProxy

```
sudo systemctl enable haproxy
sudo systemctl restart haproxy
sudo systemctl status haproxy
```

Step 6: Verify HAProxy is Listening

```
sudo ss -tlnp | grep haproxy
```

Expected Output (example):

- `*:80`
- `*:8404`

Step 7: Access HAProxy Stats Dashboard

- URL: `http://<nfs-haproxy-public-ip>:8404/stats`
- Username: `admin`
- Password: `StrongPassword@123`

1.6 Phase 3: SonarQube Server Setup

SonarQube is the code quality and security analysis server that integrates with GitHub Actions to enforce upstream quality gates before deployments.

1.6.1 SSH into SonarQube Server

```
ssh -i "your-key.pem" ubuntu@<sonarqube-public-ip>
```

Step 1: Update System

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Java 17

```
sudo apt install -y openjdk-17-jdk unzip wget net-tools
java -version
```

Step 3: Configure System Requirements for SonarQube

SonarQube uses Elasticsearch internally and requires specific kernel settings.

```
sudo bash -c 'cat >> /etc/sysctl.conf << EOF
# SonarQube / Elasticsearch Requirements
vm.max_map_count=524288
fs.file-max=131072
EOF'

sudo sysctl -p

sysctl vm.max_map_count
sysctl fs.file-max
```

Expected Output:

- `vm.max_map_count = 524288`
- `fs.file-max = 131072`

Set system limits for the `sonarqube` user:

```
sudo bash -c 'cat >> /etc/security/limits.conf << EOF
# SonarQube User Limits
sonarqube - nofile 131072
```

```
sonarqube - nproc 8192
EOF'
```

Step 4: Create SonarQube Dedicated User

SonarQube must not run as root.

```
sudo useradd -m -d /opt/sonarqube -s /bin/bash sonarqube
id sonarqube
```

Step 5: Download and Install SonarQube

```
cd /opt

sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.6.0.9
2116.zip
sudo unzip sonarqube-10.6.0.92116.zip
sudo mv sonarqube-10.6.0.92116 sonarqube
sudo chown -R sonarqube:sonarqube /opt/sonarqube

ls -la /opt/sonarqube/
```

Expected Directory Structure:

- `/opt/sonarqube/bin/`
- `/opt/sonarqube/conf/`
- `/opt/sonarqube/data/` (H2 database stored here)
- `/opt/sonarqube/logs/`
- `/opt/sonarqube/extensions/`
- `/opt/sonarqube/lib/`

Step 6: Configure SonarQube

```
sudo -u sonarqube nano /opt/sonarqube/conf/sonar.properties
```

Update/add the following properties:

```
# ==============================================================
# DATABASE CONFIGURATION
# ==============================================================
# We are using H2 - the built-in embedded database
# H2 requires NO external database setup
# Data is stored in: /opt/sonarqube/data/
# DO NOT set sonar.jdbc.url - leaving it blank = H2 is used

# ==============================================================
# WEB SERVER CONFIGURATION
```

```
# ==============================================================
sonar.web.host=0.0.0.0
sonar.web.port=9000
sonar.web.context=/

# ==============================================================
# JVM MEMORY SETTINGS (Optimized for t3.medium with 4GB RAM)
# ==============================================================
sonar.web.javaOpts=-Xmx512m -Xms256m -XX:+HeapDumpOnOutOfMemoryError
sonar.ce.javaOpts=-Xmx512m -Xms256m -XX:+HeapDumpOnOutOfMemoryError
sonar.search.javaOpts=-Xmx512m -Xms512m -XX:MaxDirectMemorySize=256m

# ==============================================================
# LOGGING
# ==============================================================
sonar.log.level=INFO
sonar.path.logs=logs
```

Note on H2 Database:

- H2 is suitable for small teams and development/testing.
- H2 data is stored in `/opt/sonarqube/data/`.
- For larger teams, migrate to PostgreSQL.

Step 7: Create Systemd Service File

```
sudo nano /etc/systemd/system/sonarqube.service
```

Add:

```
[Unit]
Description=SonarQube Code Quality Server
Documentation=https://docs.sonarqube.org
After=network.target
Wants=network.target

[Service]
Type=forking
ExecStart=/opt/sonarqube/bin/linux-x86-64/sonar.sh start
ExecStop=/opt/sonarqube/bin/linux-x86-64/sonar.sh stop
ExecReload=/opt/sonarqube/bin/linux-x86-64/sonar.sh restart
User=sonarqube
Group=sonarqube
Restart=always
RestartSec=10
LimitNOFILE=131072
LimitNPROC=8192
TimeoutStartSec=120

[Install]
WantedBy=multi-user.target
```

Step 8: Start SonarQube

```
sudo systemctl daemon-reload
sudo systemctl enable sonarqube
sudo systemctl start sonarqube
sudo systemctl status sonarqube
```

Step 9: Monitor SonarQube Startup

```
tail -f /opt/sonarqube/logs/sonar.log
tail -f /opt/sonarqube/logs/web.log
tail -f /opt/sonarqube/logs/es.log
```

Verify port 9000:

```
sudo ss -tlnp | grep 9000
```

Step 10: Access SonarQube Web UI

- URL: `http://<sonarqube-public-ip>:9000`
- Username: `admin`
- Password: `admin`

On first login, change the default password.

Step 11: Configure SonarQube for the E-Commerce Project

Step A: Create the Project

1. Login to SonarQube
2. Click **Projects** → **Create Project** → **Manually**
3. Fill in:
   - Project display name: `ecommerce-app`
   - Project key: `ecommerce-app`
   - Main branch name: `main`
4. Click **Set Up**
5. Choose **With GitHub Actions**

Step B: Generate Authentication Token

1. Click Avatar (top-right) → **My Account**
2. Click **Security** tab
3. Under **Generate Tokens**:
   - Token Name: `github-actions-token`
   - Type: Project Analysis Token
   - Project: `ecommerce-app`
   - Expiry: `90 days`

4. Click **Generate**

Critical: Copy this token immediately. It will not be shown again. Save as `SONAR_TOKEN` in GitHub Secrets.

Step C: Configure Quality Gate

1. Go to **Quality Gates**
2. Click **Create**
3. Name: `ecommerce-quality-gate`
4. Add Conditions:
   - Coverage on New Code is less than `80%`
   - Duplicated Lines on New Code is greater than `3%`
   - Reliability Rating on New Code is worse than `A`
   - Security Rating on New Code is worse than `A`
   - Maintainability Rating on New Code is worse than `A`
5. Assign:
   - Projects → `ecommerce-app` → Project Settings → Quality Gate → select `ecommerce-quality-gate`

Step 12: Verify SonarQube is Working

```
sudo systemctl status sonarqube
curl http://localhost:9000/api/system/status
```

Expected Output (example):

```
{"id":"xxx","version":"10.6.0","status":"UP"}
```

1.7 Troubleshooting Guide

Issue 1: Node Stuck in NotReady State

```
sudo journalctl -u kubelet -f --no-pager | tail -50

# FIX A: Swap is still enabled
sudo swapoff -a
sudo systemctl restart kubelet

# FIX B: ContainerD issues
sudo systemctl restart containerd
sudo systemctl restart kubelet

# FIX C: CNI plugin issues on master
kubectl delete pod -n kube-system -l k8s-app=calico-node
```

Issue 2: Worker Cannot Join — Token Expired

```
# On master node
kubeadm token create --print-join-command

# On the worker node that failed to join
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d
sudo rm -rf $HOME/.kube

sudo kubeadm join 172.31.1.10:6443 --token <new-token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Issue 3: Pods Stuck in ContainerCreating (NFS Mount Failure)

```
kubectl describe pod <pod-name> -n ecommerce

# Test NFS from the worker node
showmount -e 172.31.1.13
sudo mount -t nfs 172.31.1.13:/srv/nfs/k8s-pvs/ecommerce-products /mnt
ls /mnt
sudo umount /mnt

# If mount fails, check NFS server
sudo systemctl status nfs-kernel-server
sudo exportfs -v
sudo ss -tlnp | grep 2049
```

Issue 4: SonarQube Fails to Start

```
tail -50 /opt/sonarqube/logs/sonar.log

sudo sysctl vm.max_map_count
sudo sysctl -w vm.max_map_count=524288

ps aux | grep sonar
sudo ss -tlnp | grep 9000

java -version
```

Issue 5: HAProxy Shows Backends as DOWN

```
curl http://172.31.1.11:30080/health
curl http://172.31.1.12:30080/health

sudo tail -30 /var/log/haproxy.log
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl restart haproxy
```

Issue 6: SonarQube Quality Gate Failing in Pipeline

- Review results in SonarQube UI: `http://<sonarqube-ip>:9000` → Projects → `ecommerce-app` → Issues
- Common causes: low coverage, duplications, security hotspots
- Temporary lowering of thresholds is not recommended for production.

1.8 Useful Daily Commands Reference

```
# ── CLUSTER STATUS ──────────────────────────────
kubectl get nodes -o wide
kubectl get pods -n ecommerce -o wide
kubectl get all -n ecommerce
kubectl top nodes
kubectl top pods -n ecommerce

# ── LOGS ────────────────────────────────────────
kubectl logs <pod-name> -n ecommerce
kubectl logs <pod-name> -n ecommerce --previous
kubectl logs -f <pod-name> -n ecommerce

# ── DEBUGGING ───────────────────────────────────
kubectl describe pod <pod-name> -n ecommerce
kubectl describe node worker-node-1
kubectl get events -n ecommerce --sort-by='.lastTimestamp'

# ── EXEC INTO POD ───────────────────────────────
kubectl exec -it <pod-name> -n ecommerce -- /bin/sh

# ── DEPLOYMENT MANAGEMENT ───────────────────────
kubectl rollout status deployment/ecommerce-api -n ecommerce
kubectl rollout history deployment/ecommerce-api -n ecommerce
kubectl rollout undo deployment/ecommerce-api -n ecommerce
kubectl scale deployment ecommerce-api --replicas=3 -n ecommerce

# ── NFS VERIFICATION ────────────────────────────
showmount -e 172.31.1.13
kubectl get pv
kubectl get pvc -n ecommerce

# ── SONARQUBE ───────────────────────────────────
sudo systemctl status sonarqube
tail -f /opt/sonarqube/logs/sonar.log
curl http://localhost:9000/api/system/status

# ── HAPROXY ─────────────────────────────────────
sudo systemctl status haproxy
```

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo tail -f /var/log/haproxy.log
```

1.9 Infrastructure Summary

Component	Status Check
Kubernetes Cluster	`kubectl get nodes` (all Ready) `kubectl get pods -n kube-system` (Running)
NFS Server	`showmount -e <nfs-ip>` (lists exports) `kubectl get pvc -n ecommerce` (Bound)
HAProxy	`http://<ip>:8404/stats` (dashboard loads) `curl http://<ip>/` (app responds)
SonarQube	`http://<ip>:9000` (UI loads) `systemctl status sonarqube` (active)

SECTION 2: Source Code Management

2.1 Overview

The E-Commerce application uses GitHub as the centralized version control platform for managing source code, team collaboration, pull requests, code reviews, and release history.

All microservices source code, Kubernetes manifests, and deployment configurations are maintained in Git repositories to ensure:

- Proper version tracking
- Controlled change management
- Rollback capability
- Secure collaboration
- Streamlined development workflows

2.2 GitHub Repository Details

2.2.1 Main Application Repository

Repository URL:

https://github.com/e-commerce-application-microservice/e-commerce-app.git

Repository contains:

- Frontend source code
- Backend microservices
- API Gateway
- Dockerfiles
- CI pipeline files
- Application configuration files

2.2.2 GitOps / Deployment Repository

Repository URL:

https://github.com/e-commerce-application-microservice/ecommerce-argo-cd

Repository contains:

- Helm charts
- Kubernetes manifests
- Argo CD application definitions
- Environment values files
- Rollout strategies
- Deployment automation files

2.3 Branch Strategy

A structured branching model is implemented to support parallel development, infrastructure changes, security enhancements, and stable production releases.

2.3.1 `main`

Primary stable branch used for production-ready code.

Used for:

- Final tested releases
- Approved code changes only
- Production deployments
- Stable application versions

2.3.2 `RBAC`

Dedicated branch for Kubernetes security and access control configurations.

Used for:

- `role.yaml`
- `rolebinding.yaml`
- service accounts
- namespace permissions

Purpose: Implements Role-Based Access Control (RBAC) for secure Kubernetes operations.

2.3.3 `feature/argo-blue-green`

Feature branch used for progressive deployment strategy implementation.

Used for:

- Argo Rollouts configuration

- Blue-Green deployment manifests
- Traffic switching setup
- rollback strategy

Purpose: Enables zero-downtime deployments with controlled traffic switching.

2.3.4 `feature/ci-pipeline`

Feature branch used for Continuous Integration automation.

Used for:

- GitHub Actions workflows
- Docker image build pipeline
- Security scans
- image publishing

Purpose: Automates build, test, and image publishing workflows.

2.4 Branch Workflow

Feature branches → Pull request → Code review → `main` branch → Production deployment

2.5 Microservices Overview

The application follows a microservices architecture where each service is independently developed and deployed to handle a specific business capability.

2.6 Microservices and Port Configuration

S.No	Service Name	Type	Cluster IP	External IP	Port(s)
1	admin-service	ClusterIP	None		3014/TCP
2	api-gateway	ClusterIP	10.102.149.140		3000/TCP
3	auth-service	ClusterIP	None		3002/TCP
4	cart-service	ClusterIP	None		3006/TCP
5	ecommerce-gateway	LoadBalancer	10.104.235.220		80:32367/TCP
6	inventory-service	ClusterIP	None		3005/TCP
7	invoice-service	ClusterIP	None		3010/TCP
8	mongodb	ClusterIP	None		27017/TCP
9	order-service	ClusterIP	None		3008/TCP
10	payment-service	ClusterIP	None		3009/TCP
11	product-service	ClusterIP	None		3003/TCP
12	recommendation-service	ClusterIP	None		3013/TCP
13	review-rating-service	ClusterIP	None		3012/TCP
14	search-service	ClusterIP	None		3004/TCP
15	shipping-service	ClusterIP	None		3011/TCP
16	user-service	ClusterIP	None		3001/TCP

S.No	Service Name	Type	Cluster IP	External IP	Port(s)
17	wishlist-service	ClusterIP	None		3007/TCP

2.7 External Access Configuration

Service Name: `frontend-ui`

NodePort: `30080`

2.8 Application Architecture

2.9 Infrastructure Architecture

Developers → GitHub repositories → CI pipeline (GitHub Actions) → Container registry (GHCR) → Argo CD → Kubernetes cluster → Microservices pods → Monitoring / logging / security tools

2.10 Core Infrastructure Components

Layer	Tool / Platform	Purpose
Source Control	GitHub	Code management
CI Pipeline	GitHub Actions	Build & test automation
Container Registry	GitHub Container Registry	Store Docker images
CD Tool	Argo CD	Kubernetes deployment
Containerization	Docker	Package applications
Orchestration	Kubernetes	Run containers
Monitoring	Prometheus	Metrics collection
Dashboard	Grafana	Visualization
Logging	Grafana Loki	Centralized logs
Security	Keycloak	Authentication
Ingress / LB	HAProxy	Traffic routing

2.11 Kubernetes Cluster Infrastructure

The application is deployed in a Kubernetes cluster consisting of a Control Plane node and two Worker nodes.

Control Plane node responsibilities:

- API Server
- Scheduler
- Controller Manager
- etcd

Worker node responsibilities:

- Running application pods
- Managing services
- Rollouts
- Scaling workloads

2.12 Application Infrastructure

Frontend UI → API Gateway → 15+ backend microservices → MongoDB database

2.13 Deployment Infrastructure

Continuous Integration (CI) flow:

Code push → GitHub Actions → Build Docker image → Security scan → Push image to GHCR

Continuous Delivery (CD) flow:

Image updated → Git repository manifest update → Argo CD detects change → Sync to Kubernetes → Deployment completed

---

SECTION 3: CI Pipeline

SECTION 4: Docker, Container Registry, Trivy Image Security & SMTP Alerts

Whenever developers push code changes or manually select a service for deployment, the CI pipeline builds Docker images, performs security image scans, pushes approved images to DockerHub, and sends alerts.

This section covers Security & Image Management for our E-commerce microservices platform.

4.1 Workflow Trigger Process

4.1.1 Supported triggers (actual)

The workflow runs on:

1) **Push** to branches:

- `main`
- `master`
- `feature/ci-pipeline`

2) **Pull Request** targeting:

- `main`
- `master`

3) **Manual run** via `workflow_dispatch`

4.1.2 Manual inputs (workflow_dispatch)

Input	Type	Allowed	Purpose
`service`	choice/string	one microservice name OR `all`	Release one service independently or all services in parallel
`image_tag`	string	optional (e.g., `v1.2.0`)	Overrides auto-generated tag (highest priority)

Operational note: DevOps can release a single microservice without affecting others because the pipeline prepares a per-service matrix and builds/scans/pushes each service as an isolated artifact.

```
on:
  push:
    branches:
      - main
      - master
      - feature/ci-pipeline
  pull_request:
    branches:
      - main
      - master
  workflow_dispatch:
    inputs:
      service:
        description: "Microservice to build (or all)"
        required: true
        default: "all"
      image_tag:
        description: "Optional custom image tag (e.g., v1.2.0)"
        required: false
![](file://%7B%22source%22%3A%22attachment%3A44ae954c-5678-4de0-95c2-b93645b9f0be%3A
Screenshot_2026-04-22_125446.png%22%2C%22permissionRecord%22%3A%7B%22table%22%3A%22b
lock%22%2C%22id%22%3A%2234a446aa-8af2-80ab-ab25-d30b7aaa431d%22%2C%22spaceId%22%3A%2
2cea446aa-8af2-81b6-aadc-00030293bca4%22%7D%7D)
![](file://%7B%22source%22%3A%22attachment%3Aa472fa96-6c11-4c67-8a2b-cd59ecb592b1%3A
db14a23b-f433-460d-b31f-b13c6dbed201.png%22%2C%22permissionRecord%22%3A%7B%22table%2
2%3A%22block%22%2C%22id%22%3A%2234a446aa-8af2-80c2-b0b2-fe0fd8ee8bb6%22%2C%22spaceI
d%22%3A%22cea446aa-8af2-81b6-aadc-00030293bca4%22%7D%7D)
```

4.2 Service Matrix Preparation

4.2.1 What the “prepare matrix” job does

- If `service = all`: all microservices build in parallel.
- If a single service is selected: only that service builds.

4.2.2 Why matrix is used (release/operations)

- Reduces release time through parallelism.
- Supports the microservice architecture.
- Enables isolated release of a single service.

```
jobs:
  prepare:
```

```
    runs-on: ubuntu-latest
    outputs:
      matrix: $ steps.set-matrix.outputs.matrix
    steps:
      - id: set-matrix
        run: |
          if [ "$ github.event.inputs.service " = "all" ]; then
            echo 'matrix=["api-gateway","user-service","authentication-service","pro
duct-service","search-service","inventory-service","cart-service","wishlist-servic
e","order-service","payment-service","invoice-service","shipping-service","review-ra
ting-service","recommendation-service","admin-service"]' >> $GITHUB_OUTPUT
          else
            echo 'matrix=["$ github.event.inputs.service "]' >> $GITHUB_OUTPUT
          fi

  build_scan_push:
    needs: prepare
    strategy:
      fail-fast: false
      matrix:
        service: $ fromJson(needs.prepare.outputs.matrix)
```

4.3 Dockerfile Standardization

Each microservice has a dedicated Dockerfile located at:

`ecommerce-microservices/<service>/Dockerfile`

Examples:

- `ecommerce-microservices/product-service/Dockerfile`
- `ecommerce-microservices/cart-service/Dockerfile`
- `ecommerce-microservices/payment-service/Dockerfile`

Sample production Dockerfile (Node.js service):

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["npm","start"]
```

4.4 Dynamic Image Tagging Strategy

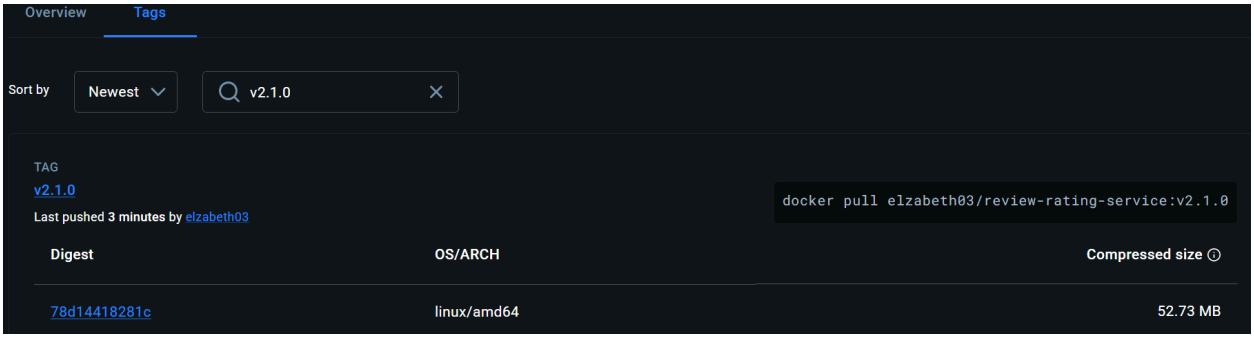
Tag resolution priority:

1. Manual custom tag input (`image_tag`)
2. Git release tag (e.g., `refs/tags/v2.0.1` → `v2.0.1`)

3. Git short commit hash (e.g., `3fa91bc`)

Final image examples:

- `mydockerhub/product-service:v1.2.0`
- `mydockerhub/product-service:v2.0.1`
- `mydockerhub/product-service:3fa91bc`
- `mydockerhub/product-service:latest`



4.5 Docker Build Process

Build pattern (per microservice):

```
docker build \
  -t DOCKER_USERNAME/product-service:<tag> \
  -f ecommerce-microservices/product-service/Dockerfile \
  ecommerce-microservices/product-service
```

Operational notes:

- Each selected microservice builds independently.

- `fail-fast: false` allows remaining services to continue even if one fails.

4.6 Image Artifact Handling

After build, the image is exported and uploaded as an artifact:

```
docker save IMAGE_NAME > image.tar
```

Later jobs download and load the same artifact:

```
docker load < image.tar
```

4.7 GitHub Secrets Management

Secrets used:

Secret	Purpose
`DOCKER_USERNAME`	DockerHub login + repository namespace
`DOCKER_PASSWORD`	DockerHub registry authentication
`SMTP_USERNAME`	SMTP sender authentication
`SMTP_PASSWORD`	SMTP password / app password
`EMAIL_RECIPIENTS`	recipients for alerts

4.8 Trivy Image Security Scan

Trivy runs after build and before push.

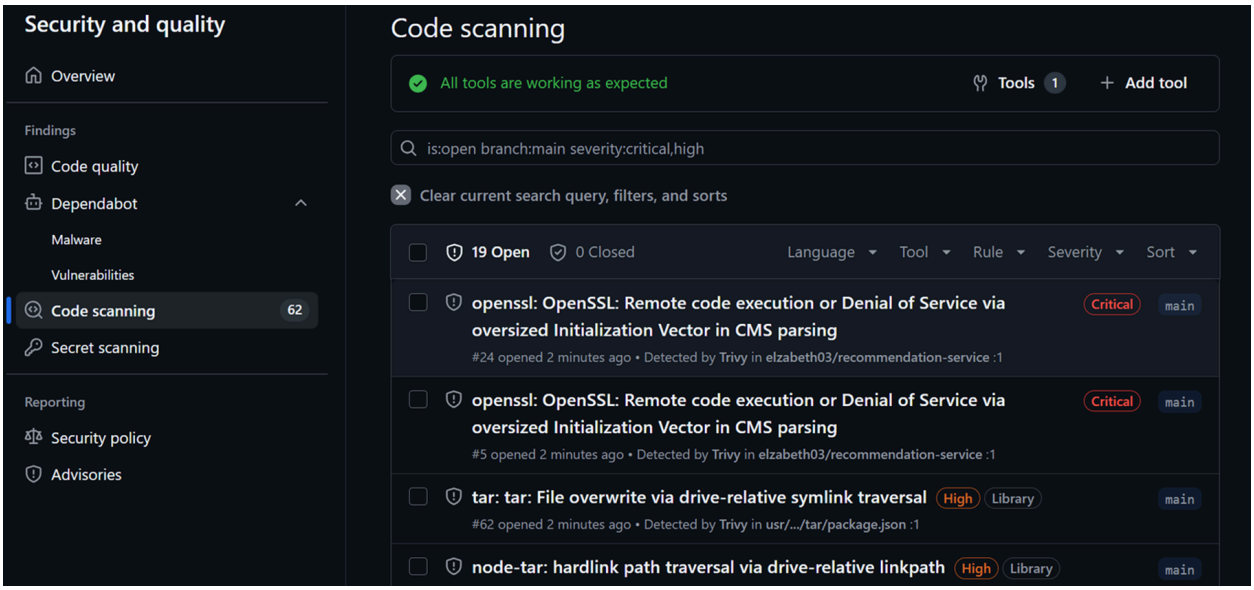
Scan settings:

- Severity: `CRITICAL,HIGH`
- Scan types: `os,library`
- Ignore unfixed: `true`
- Exit code: `1`

Sample local command:

```
trivy image mydockerhub/product-service:v1.2.0
```

If HIGH/CRITICAL vulnerabilities exist, the pipeline fails and blocks publishing.



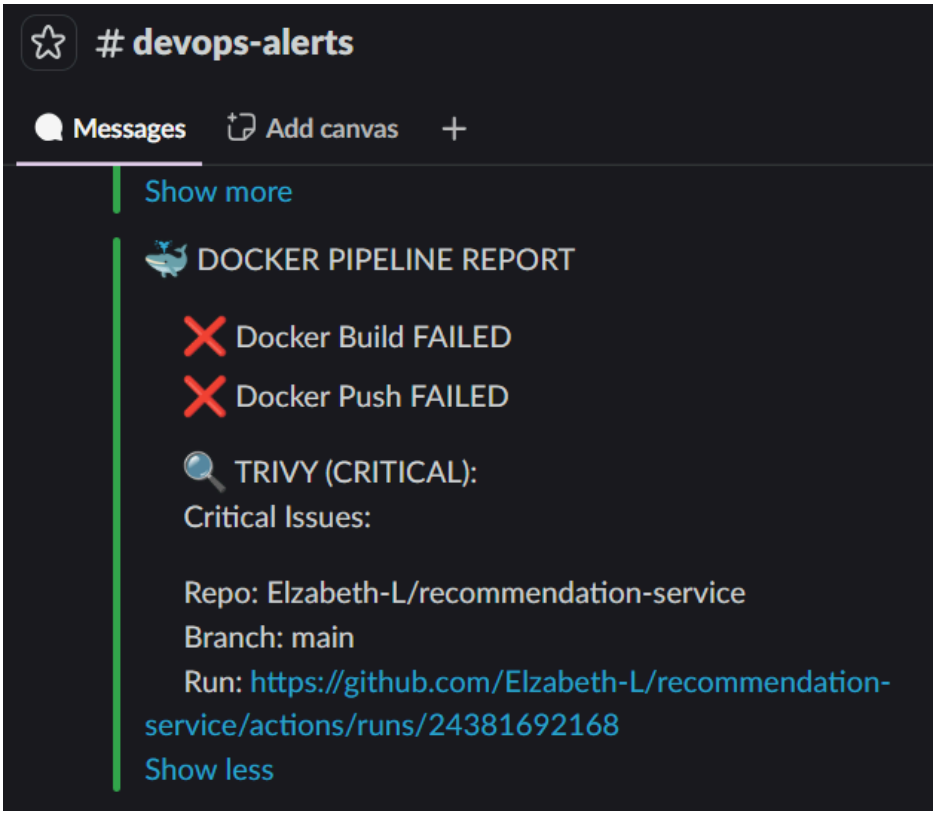
4.9 Failure Alert Notification

If Trivy fails:

- Pipeline fails for that service.
- Email alert is sent to `EMAIL_RECIPIENTS`.

Example subject:

`CI Pipeline Failed (Trivy) - product-service`



4.10 DockerHub Authentication

After successful scan:

- `docker/login-action@v3` logs into DockerHub using GitHub Secrets.

```
- name: DockerHub Login
  uses: docker/login-action@v3
  with:
    username: $ secrets.DOCKER_USERNAME
    password: $ secrets.DOCKER_PASSWORD
```

4.11 Docker Push Process

Two tags are pushed:

1) Version tag

2) `latest`

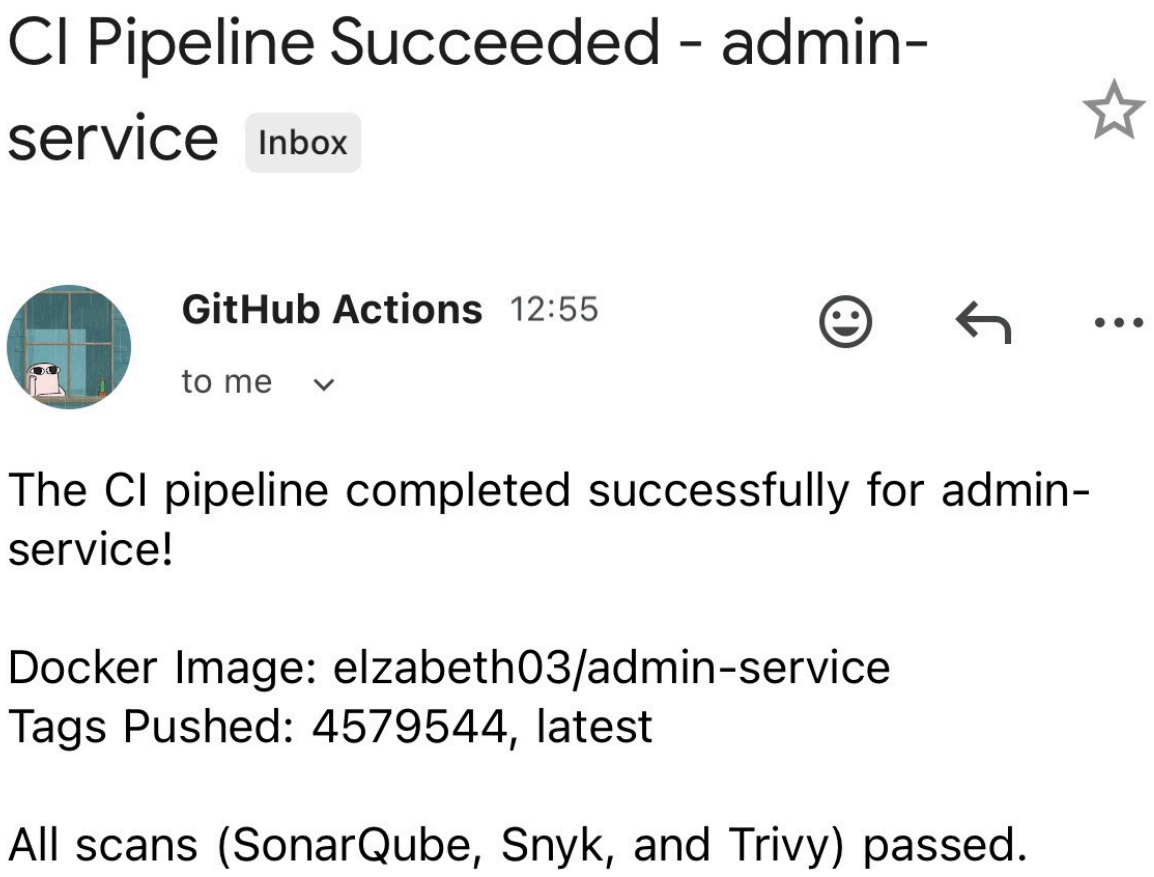
```
docker push mydockerhub/product-service:v1.2.0
docker push mydockerhub/product-service:latest
```

4.12 Final Success Notification

On successful push, a summary email is sent including:

- Service name
- Docker image
- Pushed tags
- Pipeline status

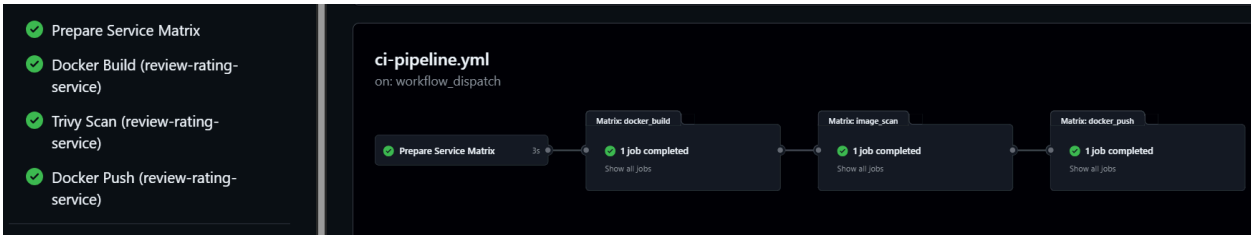
Example subject:
`CI Pipeline Succeeded - product-service`



4.13 Security Best Practices Implemented

1. Secrets stored in GitHub encrypted storage
2. No credentials in code
3. Mandatory image scan before push
4. Fail pipeline on severe vulnerabilities
5. Immutable version tagging
6. `latest` tag for active release
7. Separate image per service

8. Email alerts for failures
9. Reusable image artifacts
10. Minimal Docker base images
11. Principle of least privilege
12. Controlled registry access



4.14 Common Issues & Fixes

Issue: Docker login failed

Fix: Verify `DOCKER_USERNAME` / `DOCKER_PASSWORD`

Issue: Push denied

Fix: Confirm DockerHub repo permissions

Issue: Wrong tag generated

Fix: Verify manual `image_tag` input

Issue: Trivy failed

Fix: Patch base image OS packages or dependencies

Issue: No email received

Fix: Verify SMTP credentials and recipients

Issue: Artifact missing

Fix: Verify upload/download artifact naming consistency

4.15 End-to-End Release Flow

Developer pushes code OR manual trigger →

Prepare service matrix →

Upstream gates (SonarQube + Snyk) →

Docker Build →

Save `image.tar` →

Trivy Scan →

If fail: Email alert + stop release →

If pass: DockerHub Login →

Push image tags →

Success email →

ArgoCD consumes image for Kubernetes deployment

4.16 Interview Explanation

I was responsible for secure container image lifecycle management in our e-commerce microservices project. I implemented automated Docker builds, dynamic version tagging, image artifact transfer, Trivy vulnerability scanning, secure DockerHub authentication using GitHub Secrets, automated image publishing, and SMTP notifications for success and failure events.

4.17 Conclusion

The platform follows secure DevOps practices by automating image creation, vulnerability validation, registry authentication, versioned publishing, and release notifications. This ensures production-ready and reliable deployments.

SECTION 5: CD Pipeline

SECTION 6: Monitoring & Logging